



Bikkad IT Institute

Session 5.1: Python Tuples

Compiled by Bikkad IT Institute | Teaching Notes

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

Python Programming - Beginner Level Notes

Prepared for: Bikkad IT Institute Students

Topic: Python Tuples

Date: February 2026

1. Create a Tuple

What is a Tuple?

A tuple is an ordered, immutable collection that can store multiple items.

Key Features:

- Ordered - Items maintain their position
- Immutable - Cannot be changed after creation
- Allow duplicates
- Can contain any data type
- Defined using parentheses ()

Code Examples

Example 1: Different Ways to Create Tuple

Empty tuple

```
empty_tuple = ()
```

Tuple with values

```
numbers = (1, 2, 3, 4, 5)
```

Mixed data types

```
mixed = (1, "Python", 3.14, True)
```

Single item tuple (comma required)

```
single = (10,) # Correct
```

```
not_tuple = (10) # This is just int, not tuple
```

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

Without parentheses

```
fruits = "Apple", "Banana", "Mango"  
print(type(fruits)) # <class 'tuple'>
```

Using tuple() constructor

```
my_tuple = tuple([1, 2, 3, 4])  
print(my_tuple) # (1, 2, 3, 4)
```

From string

```
char_tuple = tuple("Python")  
print(char_tuple) # ('P', 'y', 't', 'h', 'o', 'n')
```

2. Accessing Items from a Tuple

Indexing and Slicing

Code Examples:

```
fruits = ("Apple", "Banana", "Mango", "Orange", "Grapes")
```

Positive indexing

```
print(fruits[0]) # Apple  
print(fruits[2]) # Mango
```

Negative indexing

```
print(fruits[-1]) # Grapes  
print(fruits[-2]) # Orange
```

Slicing

```
print(fruits[1:4]) # ('Banana', 'Mango', 'Orange')  
print(fruits[:3]) # ('Apple', 'Banana', 'Mango')  
print(fruits[2:]) # ('Mango', 'Orange', 'Grapes')  
print(fruits[::2]) # ('Apple', 'Mango', 'Grapes')  
print(fruits[::-1]) # Reverse tuple
```

OUR LOCATIONS

Check if item exists

```
print("Mango" in fruits) # True  
print("Grapes" not in fruits) # False
```

Find length

```
print(len(fruits)) # 5
```

3. Editing Items

Tuples are Immutable

Tuples cannot be modified after creation. Attempting to edit will cause error.

Code Examples:

Example 1: Cannot Edit (Error)

```
numbers = (1, 2, 3, 4, 5)
```

This will cause ERROR

```
numbers[0] = 100 # TypeError: 'tuple'  
object does not support item assignment
```

Example 2: Workaround - Convert to List

```
numbers = (1, 2, 3, 4, 5)
```

Convert to list, edit, convert back

```
temp_list = list(numbers)  
temp_list[0] = 100  
numbers = tuple(temp_list)  
print(numbers) # (100, 2, 3, 4, 5)
```

Example 3: Create New Tuple

```
numbers = (1, 2, 3, 4, 5)
```

Create new tuple with changes

```
numbers = (100,) + numbers[1:]  
print(numbers) # (100, 2, 3, 4, 5)
```

Example 4: Nested Mutable Objects Can Be Modified

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

Tuple with list inside

```
my_tuple = (1, 2, [3, 4, 5])
```

Can modify the list inside

```
my_tuple[2][0] = 100  
print(my_tuple) # (1, 2, [100, 4, 5])
```

But cannot reassign tuple item

```
my_tuple[2] = [10, 20] # ERROR
```

4. Adding Items

Tuples Cannot Be Modified Directly

Since tuples are immutable, you cannot add items. **Workarounds:**

Code Examples:

Example 1: Concatenation (Creates New Tuple)

```
tuple1 = (1, 2, 3)  
tuple2 = (4, 5, 6)
```

Combine tuples

```
result = tuple1 + tuple2  
print(result) # (1, 2, 3, 4, 5, 6)
```

Add single item

```
tuple1 = (1, 2, 3)  
new_tuple = tuple1 + (4,)  
print(new_tuple) # (1, 2, 3, 4)
```

Example 2: Convert to List, Add, Convert Back

```
my_tuple = (1, 2, 3)
```

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

Convert to list

```
temp_list = list(my_tuple)
```

Add items

```
temp_list.append(4)  
temp_list.extend([5, 6])
```

Convert back to tuple

```
my_tuple = tuple(temp_list)  
print(my_tuple) # (1, 2, 3, 4, 5, 6)
```

5. Deletion

Delete Operations on Tuples

Operation	Description
del tuple	Delete entire tuple
Cannot delete items	Tuples are immutable

Table 1: Deletion Operations

Code Examples:

Example 1: Cannot Delete Items (Error)

```
numbers = (1, 2, 3, 4, 5)
```

This will cause ERROR

**del numbers[0] # TypeError: 'tuple' object
doesn't support item deletion**

Example 2: Delete Entire Tuple

```
numbers = (1, 2, 3, 4, 5)
```

```
del numbers
```

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

`print(numbers)` # `NameError: name 'numbers' is not defined`

Example 3: Workaround - Convert to List

```
numbers = (1, 2, 3, 4, 5)
```

Convert to list

```
temp_list = list(numbers)
```

Remove item

```
temp_list.remove(3)
```

Convert back

```
numbers = tuple(temp_list)  
print(numbers) # (1, 2, 4, 5)
```

Example 4: Create New Tuple Without Item

```
numbers = (1, 2, 3, 4, 5)
```

Create new tuple excluding index 2

```
numbers = numbers[:2] + numbers[3:]  
print(numbers) # (1, 2, 4, 5)
```

6. Operations on a Tuple

Arithmetic Operators

Code Examples:

Concatenation (+)

```
tuple1 = (1, 2, 3)  
tuple2 = (4, 5, 6)  
result = tuple1 + tuple2  
print(result) # (1, 2, 3, 4, 5, 6)
```

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

Repetition (*)

```
numbers = (1, 2)
result = numbers * 3
print(result) # (1, 2, 1, 2, 1, 2)
```

Comparison Operators

Code Examples:

```
tuple1 = (1, 2, 3)
tuple2 = (1, 2, 3)
tuple3 = (1, 2, 4)

print(tuple1 == tuple2) # True
print(tuple1 != tuple3) # True
print(tuple1 < tuple3) # True (compares element-wise)
```

Membership Operators

Code Examples:

```
fruits = ("Apple", "Banana", "Mango")

print("Apple" in fruits) # True
print("Grapes" in fruits) # False
print("Grapes" not in fruits) # True
```

Loop Through Tuple

Code Examples:

```
fruits = ("Apple", "Banana", "Mango")
```

Using for loop

```
for fruit in fruits:
    print(fruit)
```

Using for loop with index

```
for i in range(len(fruits)):
    print(f"{i}: {fruits[i]}")
```

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

Using enumerate

```
for index, fruit in enumerate(fruits):  
    print(f'{index}: {fruit}')
```

Using while loop

```
i = 0  
while i < len(fruits):  
    print(fruits[i])  
    i += 1
```

7. Tuple Functions

Function	Description
len(tuple)	Number of items
min(tuple)	Minimum value
max(tuple)	Maximum value
tuple(iterable)	Convert to tuple
count(item)	Count occurrences
index(item)	Find index of first occurrence
sum(tuple)	Sum of all items (numeric)
sorted(tuple)	Return sorted list

Table 2: Common Tuple Functions

Code Examples:

```
numbers = (10, 50, 30, 20, 40, 20)  
  
print(len(numbers)) # 6  
print(min(numbers)) # 10  
print(max(numbers)) # 50  
print(sum(numbers)) # 170  
print(numbers.count(20)) # 2  
print(numbers.index(30)) # 2
```

sorted() returns list, not tuple

```
print(sorted(numbers)) # [10, 20, 20, 30, 40, 50]
```

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

8. The Difference Between List and Tuples

Feature	List	Tuple
Syntax	[]	()
Mutability	Mutable	Immutable
Performance	Slower	Faster
Memory	More memory	Less memory
Methods	Many methods	Few methods
Use Case	Dynamic data	Fixed data
Iteration	Slower	Faster

Table 3: List vs Tuple Comparison

Code Examples:

Example 1: Mutability Difference

List - Mutable

```
my_list = [1, 2, 3]
my_list[0] = 100
print(my_list) # [100, 2, 3]
```

Tuple - Immutable

```
my_tuple = (1, 2, 3)
```

my_tuple[0] = 100 # ERROR

Example 2: Performance Comparison

```
import time
```

List creation time

```
start = time.time()
my_list = [i for i in range(1000000)]
print("List:", time.time() - start)
```

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

Tuple creation time

```
start = time.time()
my_tuple = tuple(i for i in range(1000000))
print("Tuple:", time.time() - start)
```

Example 3: Memory Comparison

```
import sys

my_list = [1, 2, 3, 4, 5]
my_tuple = (1, 2, 3, 4, 5)

print("List size:", sys.getsizeof(my_list)) # More bytes
print("Tuple size:", sys.getsizeof(my_tuple)) # Fewer bytes
```

Example 4: Methods Available

```
my_list = [1, 2, 3]
my_tuple = (1, 2, 3)

print(dir(my_list)) # Many methods: append, extend, insert, etc.
print(dir(my_tuple)) # Only: count, index
```

9. Tuple Unpacking

What is Tuple Unpacking?

Assigning tuple items to multiple variables in one statement.

Code Examples:

Example 1: Basic Unpacking

Unpack tuple into variables

```
coordinates = (10, 20)
x, y = coordinates

print(x) # 10
print(y) # 20
```

Direct unpacking

```
name, age, city = ("Rahul", 25, "Nashik")
print(name) # Rahul
print(age) # 25
print(city) # Nashik
```

Example 2: Using Asterisk (*)

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

Unpack with remaining items

```
numbers = (1, 2, 3, 4, 5)
```

```
first, *middle, last = numbers
print(first) # 1
print(middle) # [2, 3, 4]
print(last) # 5
```

Get first two and rest

```
a, b, *rest = numbers
print(a) # 1
print(b) # 2
print(rest) # [3, 4, 5]
```

Example 3: Swap Variables

```
a = 10
b = 20
```

Swap using tuple unpacking

```
a, b = b, a
```

```
print(a) # 20
print(b) # 10
```

Example 4: Function Return Multiple Values

```
def get_student_info():
    name = "Priya"
    age = 22
    marks = 85
    return name, age, marks # Returns tuple
```

Unpack return values

```
student_name, student_age, student_marks = get_student_info()

print(student_name) # Priya
print(student_age) # 22
print(student_marks) # 85
```

Example 5: Unpacking in Loop

```
students = [
    ("Rahul", 25, "Nashik"),
    ("Priya", 22, "Mumbai"),
    ("Amit", 28, "Pune")
]
```

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

```
for name, age, city in students:  
    print(f"{name} is {age} years old from {city}")
```

10. zip() Function

What is zip()?

Combines multiple iterables into tuples.

Code Examples:

Example 1: Combine Two Tuples

```
names = ("Rahul", "Priya", "Amit")
```

```
ages = (25, 22, 28)
```

```
combined = tuple(zip(names, ages))
```

```
print(combined)
```

(('Rahul', 25), ('Priya', 22), ('Amit', 28))

Iterate through zipped tuples

```
for name, age in zip(names, ages):
```

```
    print(f"{name} is {age} years old")
```

Example 2: Combine Three Tuples

```
subjects = ("Math", "Science", "English")
```

```
marks = (85, 90, 88)
```

```
grades = ("A", "A+", "A")
```

```
result = tuple(zip(subjects, marks, grades))
```

```
print(result)
```

**(('Math', 85, 'A'), ('Science', 90, 'A+'),
('English', 88, 'A'))**

Example 3: Unzip Tuples

```
combined = (("Rahul", 25), ("Priya", 22), ("Amit", 28))
```

```
names, ages = zip(*combined)
```

```
print(names) # ('Rahul', 'Priya', 'Amit')
```

```
print(ages) # (25, 22, 28)
```

Example 4: Different Length Tuples

zip stops at shortest tuple

```
tuple1 = (1, 2, 3, 4)
tuple2 = ('a', 'b')
```

```
result = tuple(zip(tuple1, tuple2))
print(result) # ((1, 'a'), (2, 'b'))
```

Example 5: Create Dictionary from Tuples

```
keys = ("name", "age", "city")
values = ("Rahul", 25, "Nashik")
```

```
student_dict = dict(zip(keys, values))
print(student_dict)
```

{'name': 'Rahul', 'age': 25, 'city': 'Nashik'}

Practice Exercises

Exercise 1: Tuple Creation

Create a tuple with first 10 square numbers.

Exercise 2: Tuple Slicing

Given a tuple (1, 2, 3, 4, 5, 6, 7, 8, 9, 10), extract:

- First 5 elements
- Last 3 elements
- Every 2nd element
- Reverse tuple

Exercise 3: Count and Index

Count how many times 'a' appears in tuple ('a', 'b', 'a', 'c', 'a', 'd').

Exercise 4: Tuple Unpacking

Create a tuple with 5 values and unpack first, last, and middle values separately.

Exercise 5: Swap Variables

Swap three variables using tuple unpacking.

Exercise 6: Nested Tuples

Create a nested tuple representing a 3x3 matrix and access specific elements.

OUR LOCATIONS

Exercise 7: Zip Function

Combine three tuples (names, ages, cities) and print formatted output.

Exercise 8: Convert and Modify

Convert a tuple to list, add 3 new items, remove 1 item, and convert back to tuple.

Quick Reference

Operation	Syntax
Create	(1, 2, 3) or tuple()
Access	tuple[index]
Slice	tuple[start:stop:step]
Concatenate	tuple1 + tuple2
Repeat	tuple * n
Length	len(tuple)
Count	tuple.count(item)
Index	tuple.index(item)
Unpack	a, b, c = tuple
Zip	zip(tuple1, tuple2)
Delete	del tuple

Table 4: Python Tuples Quick Reference

Key Differences Summary

When to use List:

- When data needs to change
- When adding/removing items frequently
- When order matters and modification needed

When to use Tuple:

- When data should not change (immutable)
- When performance is critical
- When using as dictionary keys
- When returning multiple values from function

End of Notes

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

Prepared by: Bikkad IT Institute
For: Python Beginner Level Students
Date: February 2026

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune